

CS683 : Advanced Computer Architecture

Programming Assignment 1

Task 1 - Report

Submitted by

Ninad Narendra Kulkarni - 25M1227

Sam Navdeep Kolukula - 24b1303

Harshvardhan Parihar - 24b3980

Sairam Chari - 24b0937

CS683 - Programming Assignment 1

Task 1 Report

Task 1A - Loop Re-ordering and Unrolling

As mentioned in the hints of boilerplate code of loop reordering, Tried implementing the loop sequence as ky, kx, oy and ox.

1. Motivation : The motivation behind reordering was to take the loops which exploit the spatial locality more, inside. Ex. X and Y loops for the image. And then the kernel loops which were inside, will be taken outside. In naive code, it was jumping through rows for one image pixel. In reordering we expect it to work well because it can fetch a cache line and then use the spatial locality.
2. Considerations while implementing : As kernel loop (ky, kx) are outside, hence we can't use just one variable to accumulate (multiply kernel with pixels and sum). Also as we can't use one variable and just write it's value to the output matrix, now we are required to initialize the output matrix to 0. Also the pointer calculation is kept outside the innermost loop to avoid unnecessarily repeated instructions.
3. Effectiveness : The speedup calculated by the main.cpp (part of the source code provided) was **decreased** to About **0.43x**.

```
ninad_ubuntu@sunilkul-T480:~/MTech/cs_aca/assignment_1/task1$ ./bin/conv reorder
CS683 PA-1 Task 1 2D convolution optimization harness

=== Workload H=2048 W=2048 K=3 seed=1234 (not graded) ===
stage      correct  time(ms)  GFLOP/s   speedup
-----
naive (ref) yes       42.336    1.78      1.00x
reorder    yes       101.089   0.75      0.42x
ninad_ubuntu@sunilkul-T480:~/MTech/cs_aca/assignment_1/task1$
```

- I used the **MPKI** metric to find out the reason behind this behaviour. The MPKI for naive code was about 2.8 and for reordering it went to 8.9. As we reordered the loops, the new structure was such that the entire image is being traversed K-square time. This is the effect of the loop order (ky, kx, oy, ox). I think the reason behind this is the image working set is large to fit in the 32KB L1 cache size for my PC. That's why I think we are facing a lot of capacity misses with this reordered implementation.

Version	Instructions	L1 Loads	L1 Misses	MPKI
naive	4392971806	902957814	12337498	2.81
reorder	7073220403	1675299824	63178249	8.93

Unrolling :

On the other hand, unrolling worked decent compared to the reordering. Unrolling gave around **1.6x** speedup (running time), and about **3.3 MPKI**.

The idea with unrolling the X loop (having spatial locality) was to maximize the register reuse. In this method the four adjacent output pixels are calculated concurrently. The single kernel weight is loaded once and used four times to four different input elements. This method helps in parallel execution and also saves time. My intuition is that the superscalar nature of modern CPUs exploit this and provide slightly better performance.

```
shinad_ubuntu@sunilkul-T480:~/MTech/cs_aca/assignment_1/task1$ ./bin/conv unroll
CS683 PA-1 Task 1 2D convolution optimization harness
=== Workload H=2048 W=2048 K=3 seed=1234 (not graded) ===
stage      correct  time(ms)  GFLOP/s  speedup
-----
naive (ref) yes      46.587    1.62     1.00x
unroll     yes      28.122    2.68     1.66x
```

MPKI : unrolling helped reduce the misses (mpki) using parallel calculation.

Version	Instructions	L1 Loads	L1 Misses	MPKI
naive	4392971806	902957814	12337498	2.81
reorder	7073220403	1675299824	63178249	8.93
unroll	5780054287	1402329694	19144087	3.31

Task 1B - Tiling

Profiling : Cache sizes (command - `lscpu | grep -i cache`)

L1d cache : 64 KiB (2 instances)

L1i cache : 64 KiB (2 instances)

L2 cache : 512 KiB (2 instances)

L3 cache : 3 MiB (1 instance)

For Naive 2D convolution code :

Instructions - 4393272170

L1 loads - 903051045

L1 misses - 12306954

MPKI - 2.80

Tiling Implementation :

The tiling is implemented by dividing the image into a small grid of 32x256. The CPU processes the block completely before moving ahead. This logic helps in reducing capacity misses. In our code `bx`, `by` are the coordinates of tile which jump in multiples of the `Tile_H` and `Tile_W`.

After testing for different tile sizes, we observed that a 128x128 tile works well with more speedup.

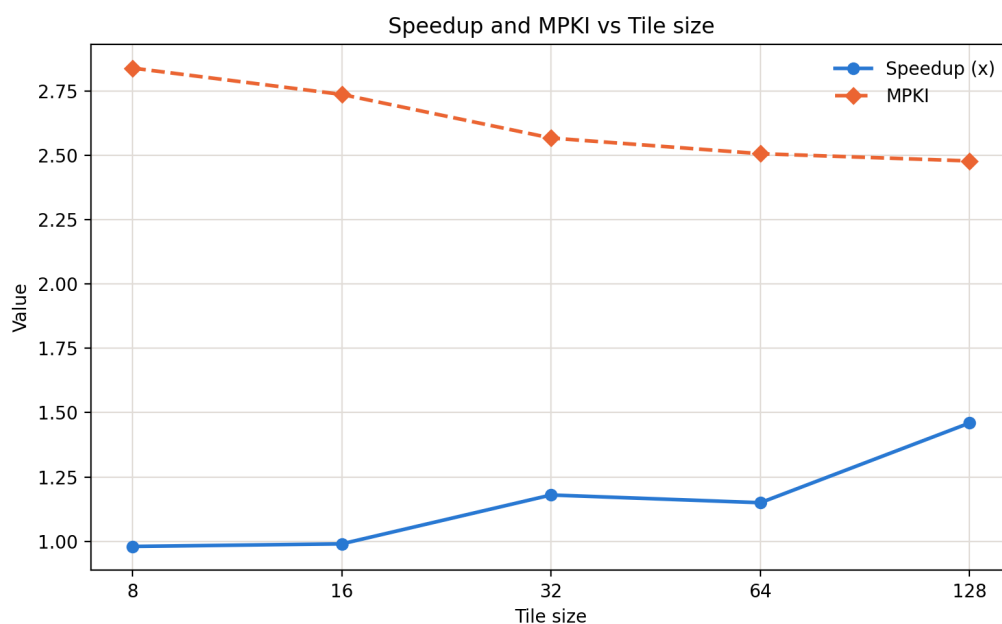
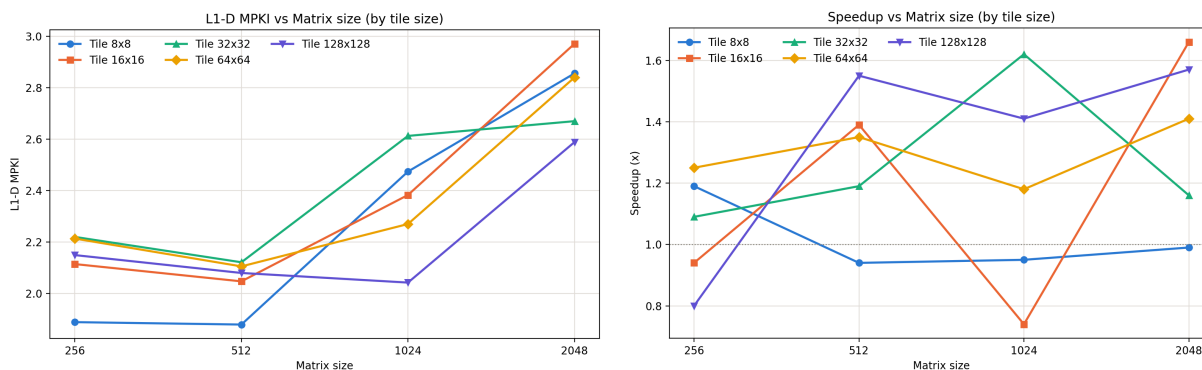


Figure : Tile Size vs Speedup and MPKI

While doing the experiment, it is found that if we set **tile_H = 32** and **tile_W = 256** then the speedup increases slightly more.

Plots -



Figures : 1. Matrix size vs L1 D MPKI, 2. Matrix size vs Speedup

Analysis :

1. **L1 D MPKI** - As observed the L1 D MPKI is reduced when we shifted to the tiling approach compared to naive approach as we can observe in *mpki_vs_matrix_size* plot above. This improvement occurs because tiling enforces cache blocking. By constraining the spatial loops to small tiles (128x128 or it can also be 32x256), the inputs, outputs and kernel matrix fits in the L1 D cache which leads to more hits. This prevents the catastrophic capacity misses that occur when sweeping a kernel weight across an entire image matrix.

2. MPKI Variation Across Matrix and Tile Sizes

As we can observe in *mpki_vs_matrix_size* plot, L1-D MPKI is lower for smaller matrix sizes. Because at these sizes, the overall memory requirement is small enough that the L2 and L3 caches store the data such that they will cover the misses of L1.

As the matrix size increases to 1024 and 2048, MPKI increases for all tiles, but slight differences between tile sizes are there.

Small Tile Overhead : Extremely small tiles like 8x8 and 16x16 causes the CPU to constantly switch between blocks, breaking sequential memory access patterns and causing the prefetcher to fetch useless data across tile boundaries, hence it increases the miss rate on large images.

3. Speedup Achievements, Factors, and Limitations

A measurable speedup was achieved for specific configurations, peaking at ~1.65x for the 16x16 tile on a 2048 matrix. The 128x128 tile provided the most consistent performance improvements (from ~1.4x to 1.58x on matrices 512 and bigger). The primary factor to this speedup is the combination of stride-1 memory access (from reordering) with L1 blocking method. However, the performance is highly volatile and limited by several factors:

Limiting Factors: As seen in the plot, the 8x8 tile failed to achieve any speedup, it is below 1.0x. Small tiles introduce massive branching; the CPU spends more cycles evaluating the outer by and bx loop conditions than doing actual calculation (floating point). Furthermore, the 16x16 tile shows severe instability. It decreases to ~0.75x at a 1024 matrix size before increasing again. This indicates that pure C++ scalar execution remains a bottleneck even when cache behavior is optimal.

Task 1C - SIMD

Number of instructions executed for naive 2D convolution - 4393534105

Number of instructions - SIMD 128 : 5255881843

Number of instructions - SIMD 256 : 4841462518

Performance Speedup :

SIMD 128 width - 2.13x

SIMD 256 width - 4.08x

Running on various matrix sizes with 128 wide SIMD registers :

Plot - Speedup vs Matrix size

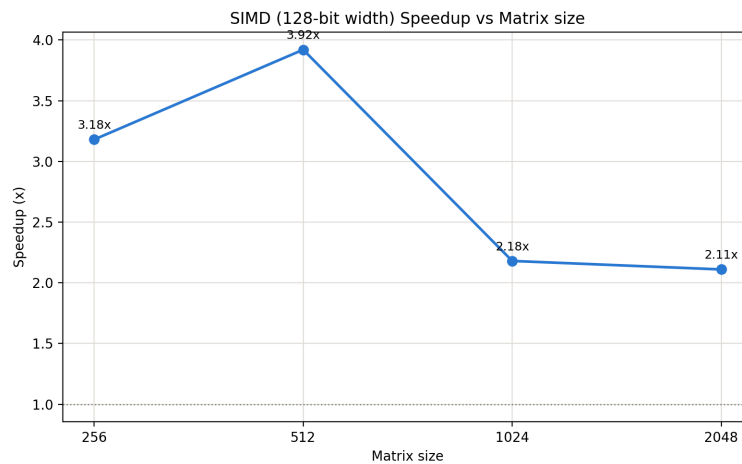


Figure : Speedup vs Matrix Size (128 width SIMD)

Running on various matrix sizes with 256 wide SIMD registers :

Plot - Speedup vs Matrix size

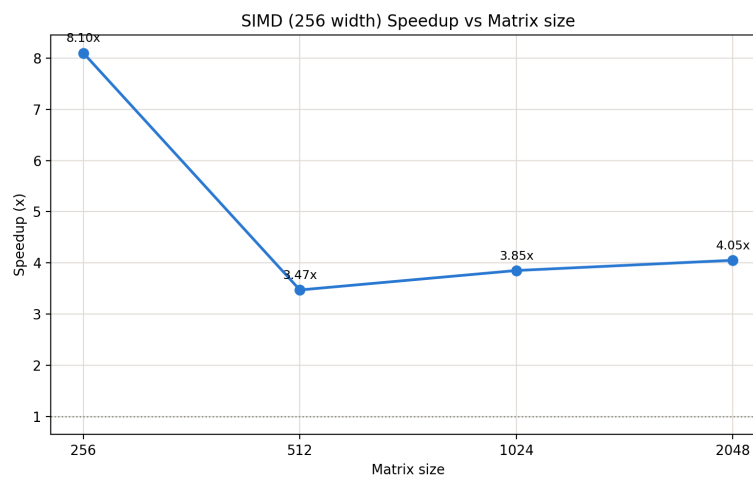


Figure : Speedup vs Matrix Size (256 width SIMD)

Analysis for Multiple Sizes of matrices using 128 and 256 wide SIMD registers :

128-bit SIMD : Achieves a peak speedup of 3.92x at matrix size 512. It matches the theoretically calculated value of 4x ($128 / 32 \text{ bit per float} = 4$).

256-bit SIMD : Delivers a peak speedup of 8.10x at matrix size 256, which also matches the theoretical limit of 8x ($256 / 32 \text{ bits per float} = 8$). The extra improvement ($>8.0x$) is given by significantly reduced loop control instructions and optimal register packing.

At matrix sizes 256 and 512, the entire input and output data fit inside the CPU's L1/L2 caches. The execution is purely compute-bound, allowing the vector execution units (FMA units) to operate at maximum efficiency without delay.

Large Matrices: 1024 to 2048 :

Performance Drop: As the matrix size scales to 1024 and 2048, speedup degrades for both implementations: 128-bit drops from 3.92x to ~2.11x, while 256-bit drops from 8.10x to ~4.05x.

Here the memory utilization is more than the size of L1/L2 cache. Hence this increases the wait time from LLC, decreasing the speedup.

SIMD 128 vs 256 - The SIMD instructions with 256 width give almost double speedup compared to the 128 wide registers.

Choosing SIMD functions -

_mm256_setzero_ps() - sets the accumulator vector to 0. No iteration through loop needed to initialize.

_mm256_loadu_ps(float*) - to fetch block of contiguous pixels from input. Here we used the unaligned version of this to take care if there are addresses that are not aligned with 16 or 32 byte boundary.

_mm256_fmadd_ps() - FMA stands for fused multiply and add. It calculates the multiplication of image with kernel and then adds it to the accumulator vector in single cycle.

_mm256_storeu_ps(float*, vector) - This is used to write the whole accumulated vector back to the output.

Task 1D - Sabka Saath, Sabka Vikaas

The `conv_optimized` implementation achieves maximum performance. Because its individual optimizations take care of different things and complement each other, eliminating bottlenecks of the CPU execution pipeline.

1. SIMD Vectorization + Register Unrolling (Hiding FMA Latency)

The 256-bit AVX2 vectorization provides 8-way Data-Level Parallelism (DLP). But the execution of consecutive FMA instructions on a single accumulator register creates a strict data-dependency chain. Because hardware FMA instructions have an execution latency of ~ 4 clock cycles, the CPU will stall waiting for the previous sum. To cater this we will use Unrolling. Unrolling the spatial loop to process 32 pixels concurrently across four independent vector registers (`acc0` through `acc3`). This allows the out-of-order execution engine to keep multiple hardware FMA pipelines fully functional for each cycle.

2. Stride-1 Access + Unaligned Vector Streaming

Vector loads (`_mm256_loadu_ps`) can be fast as the underlying memory hierarchy permits. Adjacent vector loads read sequential 32-byte chunks directly from the same 32KB L1 data cache line. By maintaining row-major, unit-stride spatial traversal across contiguous memory addresses. This avoids cache line splitting penalties.

3. Single Weight Broadcasting + Register Reuse

Inside the inner kernel loop, `_mm256_set1_ps(ker_row[kx])` broadcasts a single floating-point kernel weight into all 8 lanes of the `k_vec` register. Because this single register is reused across four consecutive vector FMA operations (`in_ptr` through `in_ptr + 24`), a single kernel weight load deals with 32 output pixels concurrently. This reduces L1 memory load traffic and saves bandwidth.

4. Hierarchical Vector Fallback

Instead of falling back to slow scalar loops for edge pixels, the implementation uses a tiered SIMD structure. This is a 32-pixel unrolled main vector loop followed by an 8-pixel standard AVX2 vector loop. This ensures that computation remains inside vector registers without branch misprediction penalties.

Together, these techniques helped us achieve around 7x speedup for the graded workload.

Speedup comparison between all implementations :

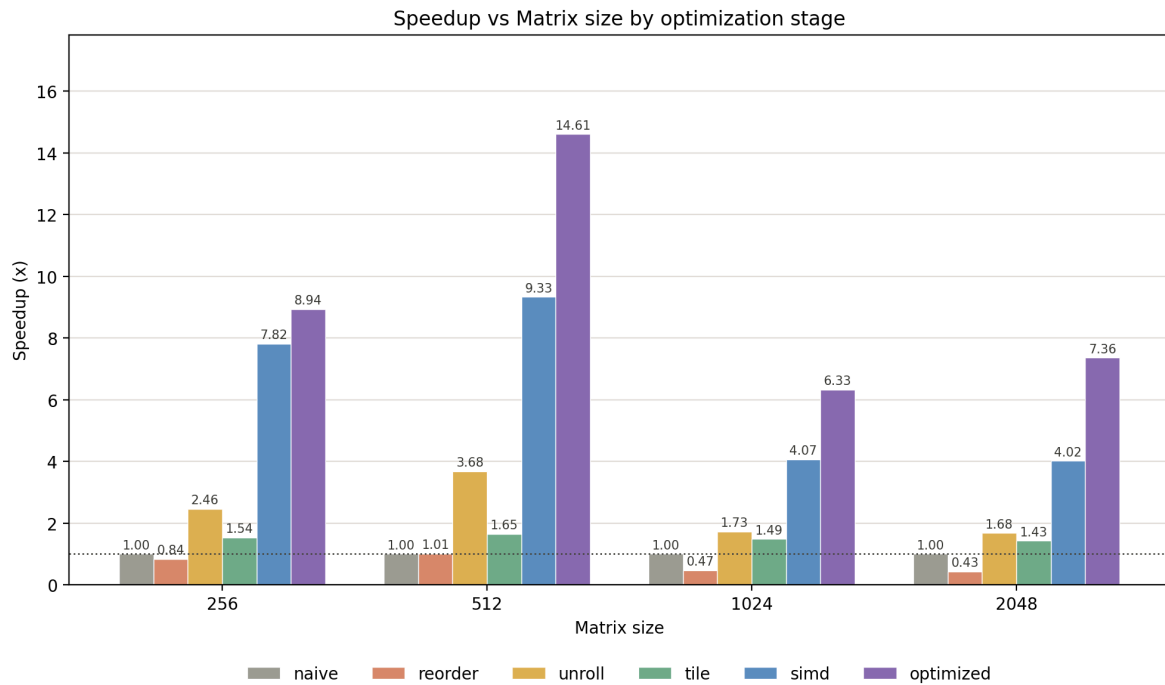


Figure : Matrix size vs Speed up (for All implementations)

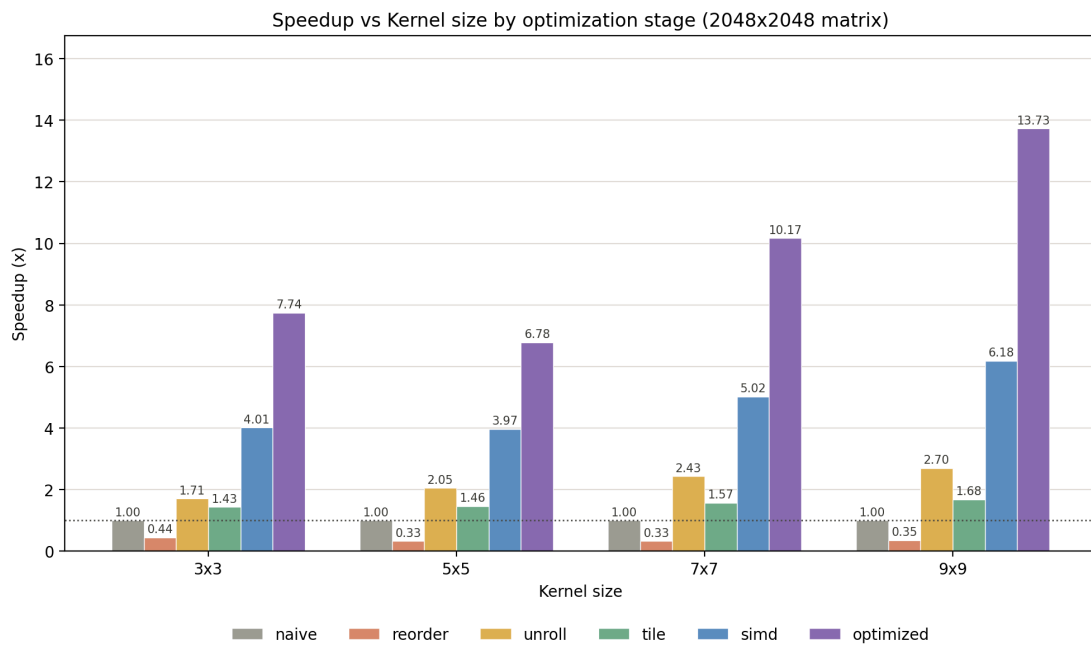


Figure : Kernel Size vs Speedup (for all implementations)

Analysis -

Performance increase for 512 size : Performance peaks at size 512 because the working set of the input, output, and kernel matrices fits entirely within the CPU's fast L1 and L2 caches. The execution allows the CPU to run without waiting for data.

Slightly lower performance for large sizes : For 1024 and 2048 sizes, the data matrices exceed L1/L2 capacity. The CPU execution pipelines stall while waiting for data to be fetched from slower L3 cache and main memory (DRAM). This memory bandwidth limits the maximum possible speedup, lowering down the SIMD and optimized scores.